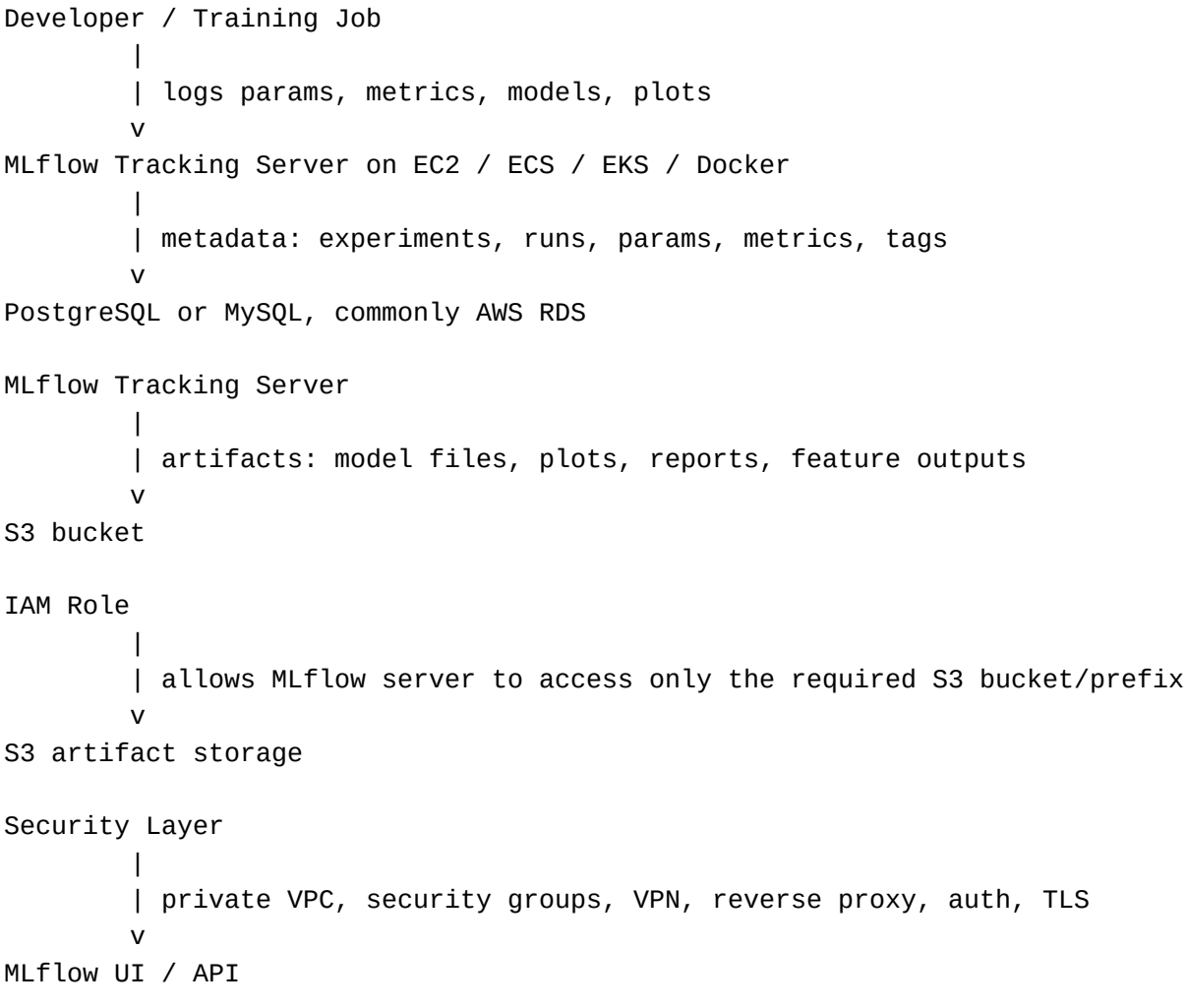
MLflow Tracking Server on AWS: Industry-Style Flow

Goal

Set up MLflow so experiments are tracked centrally, model/artifact files are stored durably in S3, metadata is stored in a database, and access is controlled securely.

High-Level Architecture



Recommended AWS Components

1. Compute
   - Simple setup: EC2 instance running MLflow.
   - More production-style: Docker container on ECS or EKS.
   - For your current learning/project flow, EC2 is easiest.

2. Backend Store
   - Use PostgreSQL or MySQL for MLflow metadata.
   - AWS managed option: RDS PostgreSQL.
   - Avoid using ./mlruns on EC2 for important work because EC2 disk is not a durable central metadata store.

3. Artifact Store
   - Use S3 for artifacts.
   - Store models, plots, evaluation reports, confusion matrices, and other large files.
   - Example artifact root:
     s3://your-mlflow-bucket/mlflow-artifacts

4. IAM
   - Attach an IAM role to the EC2 instance or container task.
   - Give permission only to the specific S3 bucket/prefix used by MLflow.
   - Avoid broad AmazonS3FullAccess in serious projects.

5. Network and Security
   - Do not expose MLflow openly to the internet unless protected.
   - Prefer private subnet access, VPN, or an authenticated reverse proxy.
   - Use security groups to restrict port 5000 or proxy ports to trusted IPs.
   - Add TLS/HTTPS when exposing a UI/API.

Step-by-Step AWS Flow

Step 1: Create an S3 bucket for MLflow artifacts
Example:
  s3://your-mlflow-bucket/mlflow-artifacts

Keep bucket versioning enabled if this is important project work.
Block public access unless there is a very specific reason not to.

Step 2: Create a database for MLflow metadata
Recommended:
  AWS RDS PostgreSQL

Example database:
  database name: mlflowdb
  user: mlflow_user
  password: stored in AWS Secrets Manager or environment variable

The backend store will hold:
  - experiment names
  - run IDs
  - params
  - metrics
  - tags
  - artifact locations

Step 3: Create IAM role for the MLflow server
Attach the role to EC2/ECS.

Minimum S3 permissions usually include:
  - s3:ListBucket on the artifact bucket
  - s3:GetObject on the artifact prefix

```
  - s3:PutObject on the artifact prefix
  - s3:DeleteObject on the artifact prefix, if you need deletion

Prefer a bucket-specific policy instead of AmazonS3FullAccess.

Step 4: Launch compute for MLflow
Simple learning setup:
  - EC2 Ubuntu instance
  - attach IAM role
  - install Python, pip, venv
  - install mlflow, boto3, psycopg2-binary

Example packages:
  pip install mlflow boto3 psycopg2-binary

Step 5: Start the MLflow server
Use PostgreSQL/MySQL as backend and S3 as artifact root.

Example command:
  mlflow server \
    --backend-store-uri postgresql://USER:PASSWORD@RDS_ENDPOINT:5432/mlflowdb \
    --default-artifact-root s3://your-mlflow-bucket/mlflow-artifacts \
    --host 0.0.0.0 \
    --port 5000

For a quick learning-only setup, this is easier but less durable:
  mlflow server \
    --backend-store-uri ./mlruns \
    --default-artifact-root s3://your-mlflow-bucket/mlflow-artifacts \
    --host 0.0.0.0 \
    --port 5000

Step 6: Protect the MLflow UI
Do not leave port 5000 open to everyone.

Safer options:
  - allow only your IP in the EC2 security group
  - access through VPN
  - put Nginx in front with authentication
  - expose through internal load balancer
  - use HTTPS/TLS

Step 7: Connect training code to MLflow
In your training/evaluation script:

  import mlflow

  mlflow.set_tracking_uri("http://YOUR_SERVER:5000")
  mlflow.set_experiment("m5-forecasting")
```

```
  with mlflow.start_run():
      mlflow.log_param("model", "lightgbm")
      mlflow.log_metric("rmse", rmse)
      mlflow.log_artifact("reports/evaluation.json")
      mlflow.sklearn.log_model(model, "model")

Step 8: Save models correctly
Common choices:
  - sklearn pipeline: .joblib or mlflow.sklearn.log_model
  - LightGBM: native model text file or mlflow.lightgbm.log_model
  - XGBoost: native model file or mlflow.xgboost.log_model
  - deep learning: framework-specific format plus MLflow flavor

Step 9: Track data separately with DVC
Use DVC for large datasets and feature tables.
Use MLflow for experiment runs, metrics, models, and evaluation artifacts.

Recommended split:
  - raw/processed data: DVC + S3/local DVC remote
  - experiments/metrics/models: MLflow
  - code: GitHub

Step 10: Production hardening checklist
  - RDS backups enabled
  - S3 versioning enabled
  - IAM least privilege
  - no public S3 bucket
  - MLflow UI protected
  - secrets not committed to Git
  - server runs as a service or container, not only inside a terminal session
  - logs sent to CloudWatch or saved clearly
  - infrastructure documented or managed with Terraform/CDK later

Best Recommendation for Your Current Situation

Phase 1: Local learning
  - Run MLflow locally with mlflow ui.
  - Log metrics and models from your scripts.

Phase 2: Good portfolio setup
  - EC2 for MLflow server.
  - S3 for artifact root.
  - SQLite or PostgreSQL backend.
  - Security group limited to your IP.

Phase 3: More industry-style setup
  - ECS or EC2 for MLflow server.
  - RDS PostgreSQL backend.
```

- S3 artifact root.
- IAM role with least privilege.
- Nginx/reverse proxy or private network access.
- HTTPS and authentication.

Common Mistakes to Avoid

1. Using only ./mlruns on EC2 for serious work.
   It is easy, but not durable enough for important metadata.

2. Giving AmazonS3FullAccess.
   It works, but it is too broad for production.

3. Opening port 5000 to the whole internet.
   MLflow should be protected.

4. Storing AWS keys in code.
   Use IAM roles or environment/secrets management.

5. Confusing DVC and MLflow.
   DVC tracks data and pipeline artifacts. MLflow tracks experiments, metrics, and models.